School of Mechanical Engineering-
Mechatronics Department

Android Based

Smart Parking System

Presented By

Koti Hemanth & Sourabh Devrukhkar

Lovely Professional University, Phagwara, India

Real-time slot monitoring · RFID-secured access · Cloud sync · Android app

TABLE OF CONTENTS

What's Inside

01 Introduction & IoT Background

Global IoT market · smart city context · problem scope

02 Problem Statement

Manual parking failure · congestion data · fuel & CO_2 stats

03 System Overview & Architecture

Components map · data flow · user/admin roles

04 Hardware Components

ATmega328P · IR sensors · EM-18 RFID · PLC + Wi-Fi

05 Software Stack

Android Studio · Arduino IDE · JavaScript · PHP · Bootstrap

06 Key System Features

RFID control · live slots · virtual wallet · admin dashboard

07 System Flow & Working

Entry/exit sequence · sensor logic · app update cycle

08 Results & Testing

Live testing outcomes · slot naming · app screens verified

09 Conclusion & Future Scope

Smart city impact · iOS/AI/camera expansions

Internet of Things & Smart Cities

What is IoT?

The Internet of Things is an interconnected network where devices communicate over the internet. It uses mobile apps, embedded frameworks, and cloud computing — and is growing rapidly across industries.

Market Scale

According to Global Data, the IoT industry is projected to reach $318 billion in new value by 2023 — reflecting its high-demand status and power to transform everyday life.

Smart Parking Need

As personal vehicle ownership rises in densely populated nations, parking difficulties have intensified — particularly near universities, government offices, and commercial complexes.

$318B

IoT Market Value

projected by 2023
(Global Data Research)

Keywords: Android application · IoT · RFID · IR sensors

02 PROBLEM STATEMENT

The Parking Crisis

10–15

minutes

Average time a driver wastes searching for a parking spot

30–40%

gridlock

Share of city traffic congestion caused by parking search

38

globe trips

Equivalent earth trips in fuel burnt by parking cruising annually

730

tons CO_2

Carbon dioxide emitted yearly from vehicles circling for parking

Before vs. After Smart Parking

❌ BEFORE — Manual System

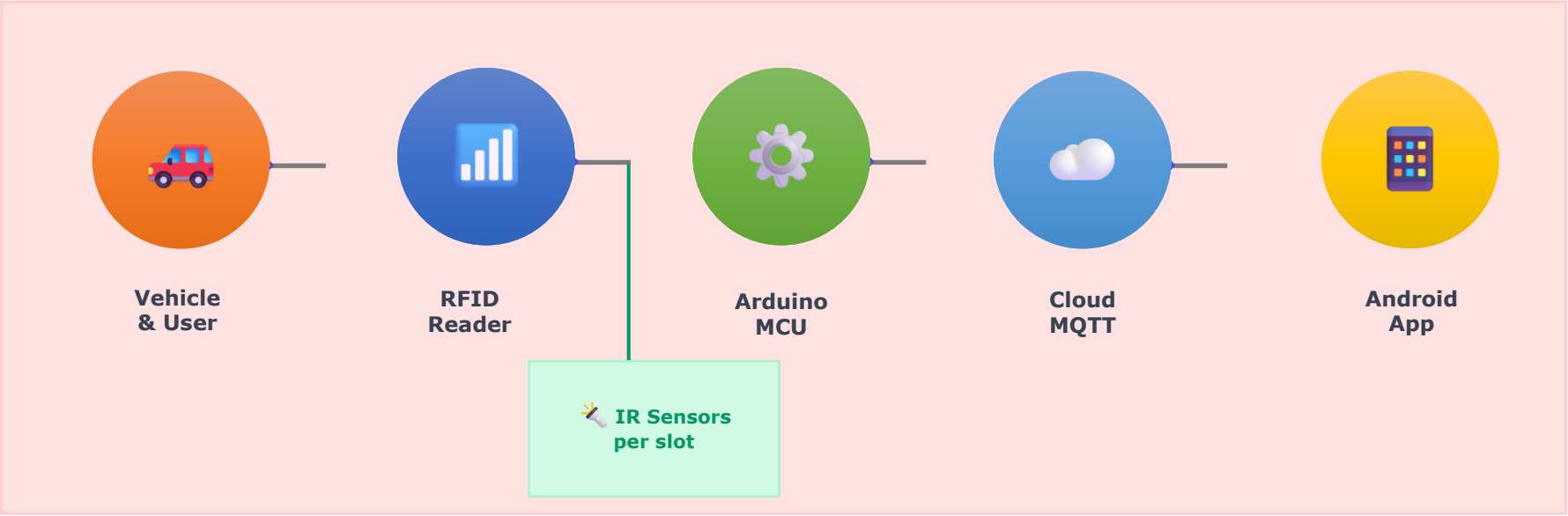
- ✗ Drivers manually search for slots using signboards
- ✗ Physical labour directs vehicles on each floor
- ✗ 10–15 minutes wasted per parking attempt
- ✗ 30–40% of city gridlock from parking search
- ✗ 38 global trips worth of fuel wasted per year
- ✗ 730 tons CO_2 emitted annually from cruising
- ✗ No real-time information — fully reactive system

VS

✅ AFTER — Smart System

- ✓ Android app shows real-time slot availability
- ✓ RFID-secured automated barrier entry & exit
- ✓ Zero wait time — reserve-and-go system
- ✓ Eliminates parking-hunt congestion entirely
- ✓ Sensors track every slot — instant cloud update
- ✓ Reduces fuel waste and CO_2 at city scale
- ✓ Fully proactive — data-driven and autonomous

Architecture at a Glance



Users

Enter via RFID card. View available slots on Android app. Tracked daily.

Admin

Add/edit parking slots. Set virtual wallet balances. Monitor usage.

Data Flow

Sensor → Arduino → Cloud (MQTT) → Android App → User decision.

Infrared Parking Sensors

How IR Sensors Work

LED Emitter

An IR LED transmits infrared light in the direction of the parking space continuously.

Photodiode Receiver

IR photodiode detects reflected infrared energy. Resistance and output voltage change with IR intensity.

Signal Interpretation

No reflection (empty space) → output 0 (Empty). Object reflects IR → output 1 (Occupied).

Arduino Controlled

Each slot's IR module connects to Arduino via VCC, GND and Signal pins. Controlled at 5V.

MODULE COMPONENTS

- **LED Emitter**
  Transmits IR light toward slot
- **Receiver Diode**
  Detects reflected IR energy

SLOT STATUS OUTPUT

0 = EMPTY
Slot available — no vehicle detected

1 = OCCUPIED
Vehicle present — slot unavailable

ATmega328P ~ The Brain

32K	8~bit	20	~20	28
Flash	**CPU**	**MHz**	**MIPS**	**pins**
Program memory	AVR architecture	Clock speed	Throughput	PDIP package

About ATmega328P

- **Atmel AVR-based 8-bit microcontroller designed for dedicated, specific function execution (not general-purpose computing)**
- **At 20MHz, some instructions complete in a single clock cycle delivering ~20 MIPS peak throughput**
- **Available in PDIP 28-pin package; compatible with 28-pin AVR development boards**
- **Part of the AVR family the most common microcontroller family alongside 8051 and PIC; chosen for capability and cost**
- **Controls all IR sensor modules, processes signals, and relays data to the cloud via Wi-Fi/PLC module**

Access Control & Data Storage

EM-18 RFID Reader Module

Frequency	125 kHz
Communication	UART / Wiegand
Card ID	12-byte unique ID
Range	Short-range NFC-like
Per User	Unique card assigned
Role	Entry + exit barrier control

Cloud / MQTT Server

Database Role

Stores parking slot states, user records, entry/exit timestamps, and daily hit counts per RFID user.

IBM MQTT / Mosquitto

Cloud MQTT server using the MQ Telemetry Transport Protocol — lightweight, machine-to-machine publish-subscribe architecture.

Flexible Scaling

Cloud flexibility allows unlimited users to be added daily. Persistent backups enable fast recovery from system failures.

Real-Time Sync

Parking data continuously updated; PLC + Wi-Fi module auto-connects and pushes state changes instantly.

Tools & Technologies

Android Studio

Mobile IDE

Builds the parking Android app. Handles different screen sizes — all devices share the same design. Creates the user-facing parking dashboard and map.

Arduino IDE

Firmware IDE

Open-source platform for writing sensor control code. Uploads C/C++ firmware to ATmega328P. Includes sample code and board configurations for beginners.

JavaScript + PHP

App Logic

App built using Apache Cordova with JavaScript as primary language. PHP manages server-side backend processing and database communication.

Bootstrap

UI Framework

Provides a responsive, consistent UI layout across all Android screen sizes. Handles styling, grid layout, and button components in the app.

Apache Cordova

App Framework

Bridges JavaScript code with native Android capabilities. Enables web-based mobile app development and secure 2-factor server communication.

RFID Access & Slot Management

RFID Entry Barrier

F1

EM-18 reader at entry. User scans RFID card. System validates ID and opens barrier after double-crossing IR sensors confirm vehicle presence.

RFID Exit Barrier

F2

Same RFID card used at exit. System logs the exit timestamp, deducts virtual wallet balance, and closes barrier after vehicle clears.

Live Slot Status

F3

Each slot continuously monitored. Status pushed to cloud database instantly. App updates in real time — green = vacant, red = occupied.

Admin Slot Edit

F4

Administrator can add, remove, or mark slots as under maintenance/reconstruction — preventing the system from directing users to unavailable spaces.

Virtual Wallet

F5

Each RFID user has a virtual credit balance. Admin controls the amount. Balance '0' blocks barrier — used to revoke parking access selectively.

Usage Tracking

F6

Cloud stores how many times each user entered/exited daily. Dashboard shows per-user session data after login — useful for billing and auditing.

The User Interface

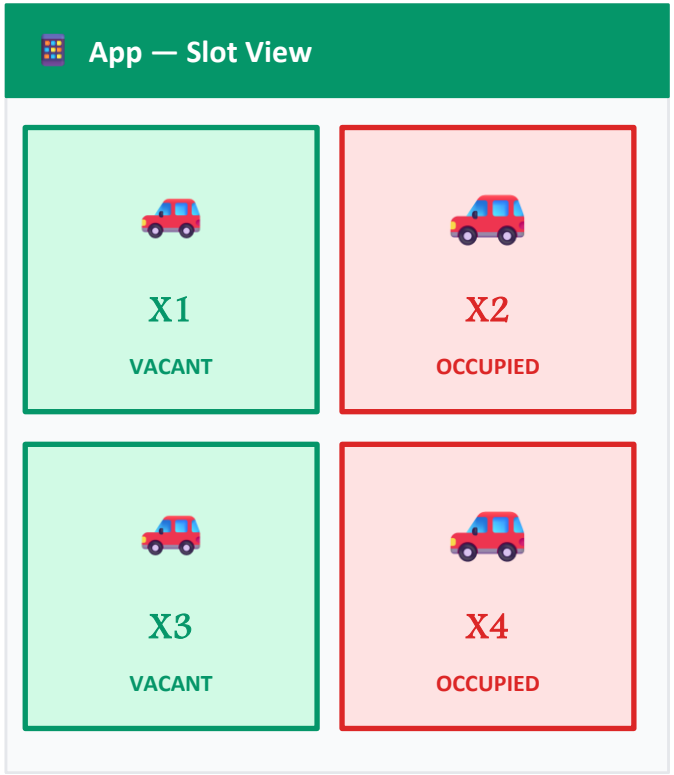
Two-Factor Login
User/admin must enter email + password. Two-factor authentication before dashboard access.

Parking Map
Visual map of the entire parking area. Slots shown as green (vacant) or red (occupied) in real time.

User Dashboard
After login, dashboard shows daily entry count, exit count, and session history for that user.

JSON Cloud API
App communicates with cloud server via secure JSON. Apache Cordova + JavaScript manage all API calls.

All Screen Sizes
Responsive Bootstrap design — same UI on phones and tablets. Built in Android Studio.



End-to-End Process

STEP 1

Arrive

Vehicle approaches parking entrance barrier

STEP 2

Scan RFID

User places RFID card on EM-18 reader at gate

STEP 3

IR Check

Double IR sensors verify vehicle presence at barrier

STEP 4

Gate Opens

Barrier opens if RFID valid + IR confirms vehicle

STEP 5

MCU Update

Arduino reads each slot IR sensor, sends data

STEP 6

Cloud Sync

PLC + Wi-Fi pushes slot states to IBM cloud via MQTT

STEP 7

App Update

Android app fetches JSON from cloud, updates slot map

STEP 8

User Sees

Driver views green/red slots, selects available space

Live System Testing Outcomes

HARDWARE & SOFTWARE USED

CATEGORY	COMPONENT
Hardware	Microcontroller (ATmega328P)
Hardware	IR Sensors (per parking slot)
Hardware	EM-18 RFID Reader Module
Hardware	Wi-Fi Module + PLC Board
Software	Android Studio
Software	Arduino IDE

✅ VERIFIED OUTCOMES

- ✓ RFID reader at entry — valid scan opens barrier
- ✓ Double IR sensor cross confirms vehicle presence
- ✓ Sensors update slot status to cloud in real time
- ✓ App shows 4 slots (X1–X4) with live green/red
- ✓ Dashboard displays daily entry/exit count per user
- ✓ Admin can edit slot configurations and wallet values
- ✓ System eliminates parking search time completely

Impact on Smart Cities

This study presents an IoT-based smart parking system using RFID, IR sensors, Arduino, cloud/MQTT, and an Android application. The system enables real-time parking availability monitoring, eliminating the need for manual search — reducing gridlock, cutting fuel waste, and lowering CO_2 emissions. A scalable solution for smart communities, universities, government offices, and large organisations.

Eliminates manual parking search

Saves 10–15 min per driver

Reduces CO_2 & fuel waste

Android app real-time access

Smart city ready solution

📌 **Scalability** · The system's cloud architecture allows unlimited users to be added. Manual stopping frameworks may be upgraded step-by-step — reducing manual labour and adding indication boards. The mobile app may be expanded to iOS and Windows for wider reach.

Future Scope & Thank You

iOS & Windows App

Expand the Android app to Apple iOS and Microsoft Windows to reach all user demographics.

Camera Integration

Attach cameras to barriers and the arm — enabling image-based vehicle recognition and spot confirmation.

Real Payment Gateway

Replace virtual wallet with actual online payment (UPI/credit card) for automated paid parking systems.

AI-Based Prediction

Use machine learning on historical data to predict slot availability and pre-allocate spaces for users.

Multi-Floor Support

Extend the system to multi-storey parking complexes with floor-level mapping in the Android app.

Push Notifications

Notify users when their preferred slot becomes available or when parking time is about to expire.